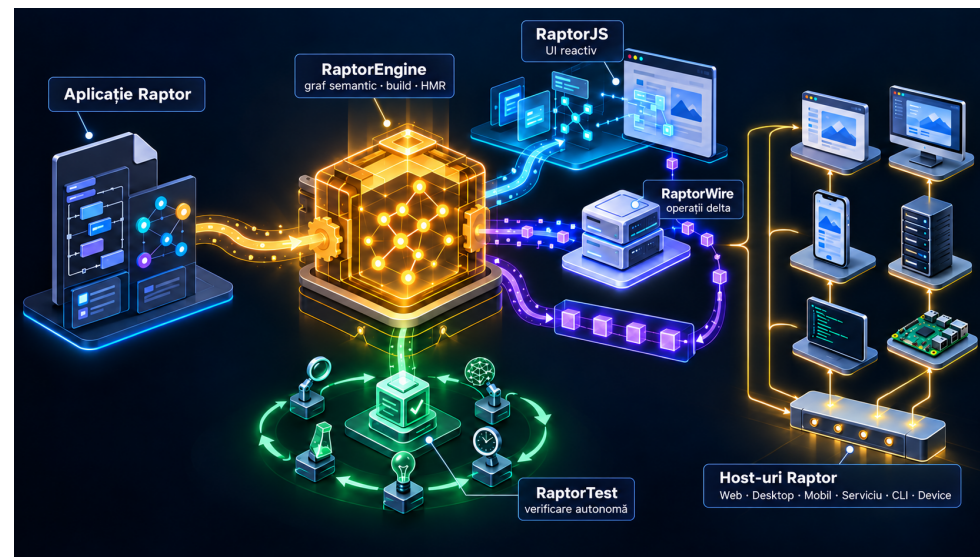


RaptorJS Ecosystem

Prezentare a tehnologiilor din monorepo

O platforma TypeScript pentru aplicatii reactive, comunicare delta, verificare comportamentala, compilare semantica si executie pe mai multe tipuri de host.



Sursa: README.md, docs/ARCHITECTURE.md si manifestele pachetelor din acest repository. Septembrie 2026.

Patru piloni folosesc aceeași schema și același graf de stare

RaptorJS

UI și reactivitate fine-grained. O schimbare actualizează numai binding-ul DOM afectat, fără Virtual DOM și fără rerendarea întregii componente.

RaptorWire

Protocol binar state-aware. După snapshot-ul inițial transmite operații semantice precum SET, INC, APPEND, PATCH sau MOVE.

RaptorTest

Teste comportamentale autonome. Observă aplicația, inferă comportamentul, sintetizează un backend digital twin, explorează stări și redă scenarii determinist.

RaptorEngine

Compiler și runtime semantic. Construiește un graf al aplicației, optimizează dependențele și generează browser, server și wire din aceeași sursă.

Pachetele care creeaza interfata si reactia exacta la schimbarea starii

@raptor/core	Nucleul reactiv: state, derived, effect, batch si untracked. Gestioneaza grafuri glitch-free, memo lazy si efecte eager.
@raptor/dom	Runtime DOM si jsx-runtime. Monteaza binding-uri fine-grained, control flow prin For si Show, plus hyperscript R fara build step.
@raptor/ui	Biblioteca de componente si primitive headless: formulare, overlay-uri, tabele, grafice, editoare, media si layout. Stilurile raman separate, fara dependente runtime.
@raptor/bundle	Bundler propriu pentru TSX si ESM. Transforma JSX, rezolva graful, face tree-shaking la nivel de modul si porneste dev server cu live reload.

Starea serverului se propaga ca operatii mici, nu ca documente complete

@raptor/wire-codec	Primitivele de codare binara: varint, zig-zag, float64, siruri si bytes length-prefixed.
@raptor/wire-core	Document versionat, snapshot-uri, opcodes, Reactive Address Space si codare adaptiva bazata pe schema.
@raptor/server	Store reactiv autoritativ. Expune query, mutation si subscription, emite snapshot plus delta si poate rula peste WebSocket.
@raptor/wire-client	Replica reactiva a clientului. Fiecare handle se comporta ca un signal; suporta loopback, WebSocket si reconnect cu delta resync.

Stratul care intelege aplicatia ca graf semantic si transforma informatia in cod de rulare

@raptor/compiler	Parseaza fisiere .raptor, construiesc Raptor IR cu stabile IDs si Semantic Application Graph. Include diferentierea grafului pentru HMR.
@raptor/engine	Orchestreaza optimizarea si codegen-ul. Include Dead Signal Elimination, Dependency Fusion, manifest de build, caching reproductibil si CLI raptor.
@raptor/run	Runtime de server pentru SSR si resume. Leaga server signals, store reactiv, RaptorWire, routing, sesiuni si observability.
@raptor/profile	Colecteaza telemetrie runtime si propune optimizari PGO: chunk folding, preload, batch sau strategie de encoding, fara sa modifice corectitudinea.

RaptorTest cauta erori prin explorarea aplicatiei, inclusiv in timp si retea

@raptor/test	Pachetul RaptorTest reunește VirtualClock, Probe, Semantic UI, Behavior Graph, Twin, Explorer, Oracle, Chaos, Replay si Coverage.
VirtualClock	Timp virtual si scheduler discret de evenimente. Acelasi scenariu se poate reproduce determinist.
RaptorTwin	Backend fals stateful, sintetizat din observatiile aplicatiei pentru a exercita fluxurile fara infrastructura reala.
Explorer si Oracle	Explorer cauta secvente de actiuni si stari; Oracle valideaza invarianti. Chaos introduce defecte controlate de retea sau raspunsuri.

Aplicatia declara ce poate face, iar runtime-ul intermediaza accesul

@raptor/runtime	Runtime de aplicatie cu capabilitati declarate: fisiere, retea, server, procese, KV, observabilitate si task-uri. Include broker, politici, revocare si delegare explicita.
@raptor/runtime-cli	CLI raptor-runtime pentru init, run, doctor, test, pack si trace. Verifica manifestul, politicile, graful static si importurile care ocolesc brokerul.
packages/runtime-native	Binar Rust care ruleaza JavaScript si TypeScript fara Node instalat. Are adapter de motor, module raptor: native si HTTP/1.1 propriu.
@raptor/forge	raptor-create genereaza proiecte pentru web, desktop si mobil, impreuna cu manifestul runtime, granita nativa si descriptorul de packaging.

Aceeasi aplicatie poate rula pe mai multe tinte printr-un contract comun

@raptor/host	Contractul comun: manifest raptor.host.json, capabilitati, bridge JS-host, lifecycle si impachetare.
Desktop si mobil	@raptor/desktop foloseste WebView-uri si gestioneaza ferestre, meniuri, notificari si instalatoare. @raptor/mobile acopera Android/iOS, navigarea, stocarea securizata si lifecycle.
Web si servicii	@raptor/web-host foloseste API-urile browserului. @raptor/service-host gestioneaza socket-uri, configuratie, secrete, health signal si drenare la SIGTERM.
CLI si device	@raptor/cli-host gestioneaza TTY, argumente si confirmari. @raptor/device-host declara pini si magistrale, somn, OTA rollback si watchdog.

Un traseu tipic al unei aplicatii Raptor

1. Autor	Scrie UI si stare in TSX sau .raptor, folosind RaptorJS.
2. Compilare	RaptorEngine construiește graful semantic; RaptorBundle sau engine-ul produce cod pentru target.
3. Executie	RaptorRun sau RaptorRuntime porneste aplicatia cu capabilitatile declarate.
4. Realtime	Serverul autoritativ emite operatii RaptorWire; clientul actualizeaza signal-ul si binding-ul DOM afectat.
5. Calitate	RaptorTest exploreaza comportamentul si poate reda deterministic orice scenariu descoperit.
6. Livrare	Un adaptor host impacheteaza aceeasi aplicatie pentru web, desktop, mobil, serviciu, CLI sau device.

Mesaj de retinut

Raptor foloseste aceeasi intelegere a starii aplicatiei in compiler, runtime, retea, testare si UI. Aceasta reduce munca redundanta si permite actualizari precise, transport compact si executie portabila.

Repository analizat: raptorjs. Document de prezentare intern.